BÖLÜM 4 / MODULE 4

İleri Düzey Senkronizasyon & Kilit-Siz Araçlar

Advanced Synchronization & Lock-Free Primitives

StampedLock iyimser okuma mekanizması, CountDownLatch tek seferlik kapılar, CyclicBarrier tekrar kullanılabilir buluşma noktaları, Semaphore kaynak havuzları, Exchanger ve dinamik Phaser mimarisi.

StampedLock optimistic reading stamps, CountDownLatch fan-out gates, CyclicBarrier reusable rendezvous, Semaphore resource pools, pairwise Exchanger, and dynamic multi-phase Phaser.

19

BÖLÜM: İLERI DÜZEY SENKRONIZASYON & KILIT-SIZ ARAÇLAR | MODULE: ADVANCED SYNCHRONIZATION & LOCK-FREE PRIMITIVES

StampedLock: İyimser Okuma (Optimistic Reading) & Stamp Doğrulama

StampedLock: Optimistic Reading, Stamp Validation & Upgrades

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
                STAMPEDLOCK: İYİMSER OKUMA (OPTIMISTIC READ) DÖNGÜSÜ

 1. İYİMSER OKUMA BAŞLAT:
    long stamp = sl.tryOptimisticRead(); // SIFIR KİLİT! Bellek durumu snapshot'ı al.

 2. YEREL DEĞİŞKENLERE KOPYALA:
    double currentX = x;
    double currentY = y;

 3. STAMP'I DOĞRULA (validate):
    if (!sl.validate(stamp)) { ──► "Biri araya girip yazdı mı?"
        // EVET! Araya bir yazıcı girdi, veriler tutarsız olabilir!
        // PES ETME: Karamsar (Pessimistic) Okuma Kilidine geç:
        stamp = sl.readLock();
        try {
            currentX = x;
            currentY = y;
        } finally {
            sl.unlockRead(stamp);
        }
    }
 AVANTAJ: Yazıcı yoksa %100 KİLİTSİZ ve sıfır CAS maliyetiyle okuma tamamlanır!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir müzedeki tarihi bir saatin akrep ve yelkovanına bakmak istiyorsunuz. Normal sistemde saatçiyi durdurup saati kilitlersiniz. StampedLock'un 'İyimser Okuması'nda ise saati hiç kilitlemezsiniz! Yanından geçerken akrep ile yelkovana hızlıca göz ucuyla bakarsınız (tryOptimisticRead). Sonra elinizdeki bilete bakıp sorarsınız: 'Ben bakarken saatçi saati kurcaladı mı?' (validate). Saatçi dokunmadıysa saati öğrendiniz, bitti! Dokunduysa hemen geri döner, bu sefer saati durdurup garantiye alarak okursunuz!

🇬🇧 ENGLISH (ELI5)

Imagine checking the time on a grand clock. Under standard locking, you halt the clockwork mechanism while looking. StampedLock's 'Optimistic Read' never halts the clockwork! You glance at the hands as you walk past (`tryOptimisticRead`). Then you check your ticket validation stamp: 'Did the clockmaker wind the hands while I was glancing?' (`validate`). If not, your read is mathematically valid with ZERO lock overhead! If the stamp invalidated, you gracefully fall back to a full pessimistic lock.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

StampedLock Mimarisi ve Reentrancy Uyarısı: 1) İyimser Okuma (Optimistic Read): `tryOptimisticRead()` bir kilit almaz; yalnızca sürüm numarasını (Stamp) döner. Okuma sırasında herhangi bir yazıcı thread `writeLock()` alırsa stamp sürümü değişir ve `validate(stamp)` false döner. 2) ReentrantReadWriteLock'tan Farkı: RWLock'ta okuyucular yazıcıları tamamen bloke eder; bu da 'Writer Starvation' (yazıcıların aç kalması) sorununa yol açar. StampedLock'ta iyimser okuyucular yazıcıları ASLA bloke etmez! 3) ⚠️ EN BÜYÜK TUZAK (Reentrant DEĞİLDİR!): StampedLock `Reentrant` DEĞİLDİR! Bir thread tuttuğu `readLock()` varken tekrar `readLock()` almaya çalışırsa veya kilit içinde kendini çağıran metotlar varsa DEADLOCK meydana gelir!

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

StampedLock Internal Topology: 1) Stamp Encoding: StampedLock encodes lock state as a 64-bit integer combining sequence bits and reader counts. An optimistic read returns the current sequence stamp with zero memory writes. 2) Writer Starvation Immunity: In `ReentrantReadWriteLock`, continuous readers starve pending writers indefinitely. StampedLock allows writers to acquire the exclusive lock instantly, invalidating optimistic readers without stalling. 3) ⚠️ Critical Fatal Constraint: StampedLock is NON-REENTRANT! If a thread holding a read or write lock calls another method attempting to acquire the same StampedLock, it deadlocks itself immediately. It also does NOT implement the standard `Lock` interface.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: StampedLock ile İyimser Okumalı 2D Nokta Modeli
import java.util.concurrent.locks.StampedLock;

public class Point2D {
    private double x, y;
    private final StampedLock sl = new StampedLock();

    public void move(double deltaX, double deltaY) {
        long stamp = sl.writeLock(); // Karamsar Yazma Kilidi
        try {
            x += deltaX;
            y += deltaY;
        } finally {
            sl.unlockWrite(stamp);
        }
    }

    public double distanceFromOrigin() {
        // 1. İyimser Okuma Dene (Kilit YOK!)
        long stamp = sl.tryOptimisticRead();
        double curX = x, curY = y;

        // 2. Araya yazıcı girdi mi kontrol et
        if (!sl.validate(stamp)) {
            // Yazıcı girmiş! Karamsar okuma kilidine düş:
            stamp = sl.readLock();
            try {
                curX = x;
                curY = y;
            } finally {
                sl.unlockRead(stamp);
            }
        }
        return Math.sqrt(curX * curX + curY * curY);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakatın 1 Numaralı StampedLock Sorusu: 'StampedLock reentrant mıdır?' Yanıt: 'HAYIR! Kesinlikle reentrant değildir. Aynı thread aynı kilide ikinci kez girmeye çalışırsa anında self-deadlock yaşanır!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Mandatory StampedLock Interview Check: 'Is StampedLock reentrant?' Senior Answer: 'NO! StampedLock is strictly NON-REENTRANT. Attempting reentrant acquisition on the same thread will permanently deadlock the application.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- StampedLock iyimser okumada hiçbir kilit almaz; yazıcıları asla bloke etmez.
- validate(stamp) başarısız olursa karamsar readLock'a geri dönülür.
- StampedLock REENTRANT DEĞİLDİR; kendini çağıran kilitler deadlock olur.

🎯 Key Takeaways for the Interview

- Optimistic reads in StampedLock acquire no lock and never block writers.
- If validate() fails, execution falls back gracefully to a pessimistic readLock.
- StampedLock is NOT reentrant; self-acquisitions cause instant deadlocks.

20

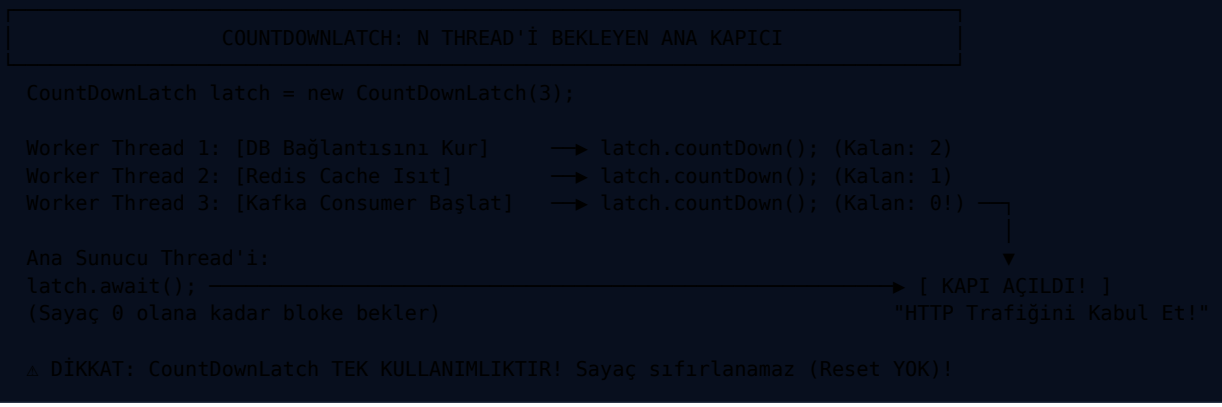
BÖLÜM: İLERI DÜZEY SENKRONIZASYON & KILIT-SIZ ARAÇLAR | MODULE: ADVANCED SYNCHRONIZATION & LOCK-FREE PRIMITIVES

CountDownLatch: Tek Seferlik Geri Sayım Kapıları & Dağıtık Fan-Out

CountDownLatch: One-Shot Countdown Gates for Distributed Fan-Out

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir roket fırlatması düşünün. Ana kumandanın (Main Thread) 'ATEŞLE' düğmesine basması için 3 uzmanın yeşil ışık yakması gerekir: 1. Yakıt uzmanı onaylar (countDown), 2. Motor uzmanı onaylar (countDown), 3. Rota uzmanı onaylar (countDown). Sayaç 3'ten geriye sayar. Sayaç sıfıra ulaştığı anda fırlatma kapısı açılır (`await()` biter) ve roket uzaya fırlar! Bu kilit tek kullanımlıktır; bir kere sıfıra indi mi bir daha kurulamaz!

🇬🇧 ENGLISH (ELI5)

Think of a rocket launch protocol. The mission commander (`main` thread) holds their finger over the ignition switch (`latch.await()`). Three independent engineers must report ready: Telemetry (`countDown()`), Propulsion (`countDown()`), and Life Support (`countDown()`). As the latch decrements from 3 to 0, the gate springs open, releasing the commander to initiate liftoff. CountDownLatch is strictly single-use; once the count hits zero, it cannot be reset.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

CountDownLatch ve AQS Paylaşımlı Durum Mimarisi: 1) AQS Shared Mode: Latch, AQS'in `acquireSharedInterruptibly` ve `releaseShared` metotlarını kullanır. İlk sayaç AQS'in `state` değişkenine yazılır. 2) countDown() Atomikliği: Her `countDown()` çağrısı `state` değişkenini CAS ile 1 azaltır. Sayaç tam 0 olduğu anda, AQS bekleme kuyruğundaki tüm thread'leri `LockSupport.unpark()` ile aynı anda uyandırır. 3) Fan-Out / Fan-In Deseni: MAANG mikromimarisinde bir ana API isteği geldiğinde 5 farklı alt servise (Fiyat, Stok, Kullanıcı, Öneri, Kargo) paralel istek atılır. Ana thread `latch.await(500, TimeUnit.MILLISECONDS)` ile bekler; tüm servisler yanıt verince sonuçlar birleştirilir (Fan-In).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

CountDownLatch Mechanics in AQS: 1) Shared Synchronizer: Unlike ReentrantLock which acquires exclusive state, CountDownLatch employs AQS Shared Mode. The initial constructor parameter sets the internal AQS `state`. 2) Atomic Decrement: `countDown()` executes CAS loops decrementing `state`. When `state` transitions to 0, it calls `doReleaseShared()`, cascading down the CLH queue unparking every thread blocked on `await()`. 3) One-Shot Invariant: Once `state == 0`, subsequent calls to `countDown()` are inert NOOPs, and calls to `await()` return immediately. It cannot be rewound; reusable synchronization requires `CyclicBarrier`.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: CountDownLatch ile Paralel Mikroservis Fan-Out / Fan-In
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.TimeUnit;

public class MicroserviceAggregator {
    void main() throws InterruptedException {
        int serviceCount = 3;
        var latch = new CountDownLatch(serviceCount);

        IO.println("Ana Servis: Alt servis çağrıları başlatılıyor...");

        // 3 paralel servis çağrısı başlat
        Thread.ofPlatform().name("PriceService").start(() -> {
            try { Thread.sleep(200); IO.println("✔ Fiyat Servisi Tamamlandı"); }
            catch (Exception ignored) {}
            finally { latch.countDown(); }
        });

        Thread.ofPlatform().name("InventoryService").start(() -> {
            try { Thread.sleep(150); IO.println("✔ Stok Servisi Tamamlandı"); }
            catch (Exception ignored) {}
            finally { latch.countDown(); }
        });

        Thread.ofPlatform().name("RecommendationService").start(() -> {
            try { Thread.sleep(300); IO.println("✔ Öneri Servisi Tamamlandı"); }
            catch (Exception ignored) {}
            finally { latch.countDown(); }
        });

        // Maksimum 500ms bekle (SLA Koruması)
        boolean allDone = latch.await(500, TimeUnit.MILLISECONDS);
        if (allDone) {
            IO.println("🎉 Tüm alt servisler yanıt verdi! Yanıt birleştirildi.");
        } else {
            IO.println("⚠ Bazı servisler zaman aşımına uğradı, fallback yanıtı dönülüyor!");
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Kritik Mülakat Hatası: `countDown()` metodunu `try-finally` bloğu içine almamak! Eğer worker thread içinde beklenmedik bir RuntimeException fırlarsa ve `countDown()` çağrılamazsa, ana thread sonsuza dek `await()`te asılı kalır! Kural: `try { ... } finally { latch.countDown(); }`!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Critical Interview Pitfall: Omitting `latch.countDown()` from a `finally` block! If a worker thread throws an unhandled RuntimeException before reaching `countDown()`, the counter never decrements to zero, hanging the coordinating thread permanently.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- CountDownLatch tek kullanımlıktır; sayaç sıfıra ulaştığında bir daha sıfırlanamaz.
- countDown() çağrıları thread çökmelerine karşı DAİMA finally içinde olmalıdır.
- Mikroservis paralel sorgulamalarında (Fan-Out/Fan-In) zaman aşımlı await kullanılır.

🎯 Key Takeaways for the Interview

- CountDownLatch is strictly single-use and cannot be reset once drained.
- Always wrap countDown() invocations inside a finally block to prevent hangs.
- Use await(timeout) for resilient SLA enforcement in parallel fan-out architectures.

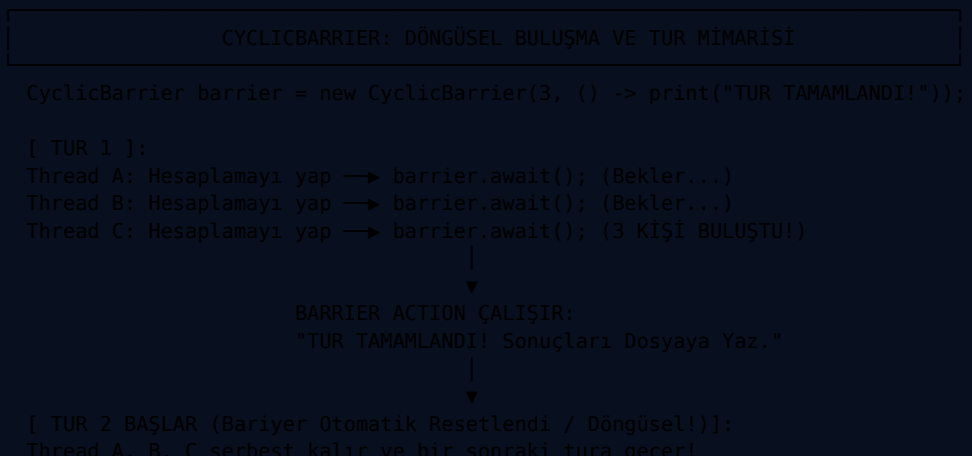
21

BÖLÜM: İLERI DÜZEY SENKRONIZASYON & KILIT-SIZ ARAÇLAR | MODULE: ADVANCED SYNCHRONIZATION & LOCK-FREE PRIMITIVES

CyclicBarrier: Tekrar Kullanılabilir Buluşma Noktaları & Bariyer Eylemi

CyclicBarrier: Reusable Multi-Thread Rendezvous & Barrier Actions

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir lunaparktaki 4 kişilik gondol düşünün. Gondol 4 kişi dolmadan ASLA hareket etmez. İlk binen 3 kişi gondolun içinde oturup 4. kişiyi bekler (`await()`). 4. kişi bindiği anda kilitler kapanır ve gondol sallanır (Barrier Action). Tur bitince herkes iner ve gondol TEK-RAR boşalır; sonraki 4 kişi için baştan kullanılır (Cyclic / Döngüsel)! CountDownLatch gibi çöpe atılmaz!

🇬🇧 ENGLISH (ELI5)

Picture a 4-person whitewater rafting crew. The raft cannot depart until all 4 rowers take their seats. The first three climbers sit with paddles ready (`await()`). The instant the fourth rower steps aboard, the team pushes off down the rapids together (Barrier Action). At the bottom of the river, the raft is carried back up and re-used for the next round! Unlike single-use countdown latches, CyclicBarrier automatically resets for subsequent cycles.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

CyclicBarrier ve CountDownLatch Farkları: 1) Tekrar Kullanılabilirlik (Cyclic): CyclicBarrier, tüm thread'ler buluştuğunda iç sayacını otomatik olarak ilk değerine geri sıfırlar. Matris çarpımları, genetik algoritmalar ve çok adımlı paralel simülasyonlar için biçilmiş kaftandır. 2) Barrier Action: Tüm thread'ler bariyere ulaştığında, içlerinden rastgele bir tanesi verilen `Runnable` görevini (ör. tur sonuçlarını diskte birleştirme) tek başına çalıştırır; bitince tüm thread'ler bir sonraki adıma serbest bırakılır. 3) Kırık Bariyer (BrokenBarrierException): Eğer bekleyen thread'lerden biri zaman aşımına uğrar veya `interrupt` edilirse, tüm bariyer kırılır (Broken state). Diğer tüm bekleyen thread'ler `BrokenBarrierException` alarak çöker; böylece eksik veriyle yanlış işlem yapılması donanımsal olarak engellenir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

CyclicBarrier Mechanics & Failure Handling: 1) Cyclic Reset: CyclicBarrier resets its internal count automatically once the parties count reaches zero, advancing to a new 'Generation'. It is specifically tuned for iterative algorithms (e.g. parallel Jacobi relaxation or map-reduce matrix passes). 2) Barrier Action Execution: When the last thread trips the barrier, it synchronously executes the barrier action Runnable on its own stack before releasing the awaiting parties. 3) Broken Barriers: If any thread in the barrier is interrupted or times out, the barrier enters the 'Broken' state, unparking all other threads with a `BrokenBarrierException` to prevent deadlock and state corruption.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Çok Turlu Paralel Simülasyonda CyclicBarrier Kullanımı
import java.util.concurrent.CyclicBarrier;

public class ParallelMatrixSimulation {
    void main() {
        int workerCount = 3;

        // 3 işçi buluştuğunda çalışacak ortak bariyer eylemi
        var barrier = new CyclicBarrier(workerCount, () -> {
            IO.println("=== TUR BİTTİ: Ara sonuçlar birleştirildi. Yeni tura geçiliyor! ===\n");
        });

        for (int i = 1; i <= workerCount; i++) {
            final int id = i;
            Thread.ofPlatform().name("Worker-" + id).start(() -> {
                try {
                    for (int round = 1; round <= 2; round++) {
                        IO.println("İşçi " + id + " | Tur " + round + " hesaplıyor...");
                        Thread.sleep(100 * id); // Farklı hızlarda hesaplarlar

                        IO.println("İşçi " + id + " bariyere vardı, diğerlerini bekliyor...");
                        barrier.await(); // 3 kişi tamamlanana kadar bekle
                    }
                } catch (Exception e) {
                    Thread.currentThread().interrupt();
                }
            });
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'CountDownLatch ile CyclicBarrier arasındaki en temel iki fark nedir?' diye sorarsa: '1) CountDownLatch tek kullanımlıktır, CyclicBarrier sonsuz döngüde tekrar kullanılabilir (resetlenir). 2) CountDownLatch thread'lerin bir olayı beklemesini sağlar; CyclicBarrier ise thread'lerin birbirini beklemesini sağlar!' denmelidir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Top MAANG Difference Question: 'What are the two foundational differences between CountDownLatch and CyclicBarrier?' Senior Answer: '1) CountDownLatch is single-use and cannot be reset; CyclicBarrier is cyclic and auto-resets per generation. 2) CountDownLatch waits for events/tasks to finish (threads don't wait for each other); CyclicBarrier forces concurrent threads to wait for each other at a mutual rendezvous.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- CyclicBarrier her turda kendini sıfırlayarak sonsuz kez tekrar kullanılabilir.
- Barrier Action, son gelen thread tarafından tek başına yürütülür.
- Bir thread kesilirse bariyer Broken state'e geçer ve tüm thread'leri korur.

🎯 Key Takeaways for the Interview

- CyclicBarrier auto-resets its count, enabling iterative multi-phase processing.
- The barrier action is executed by the final arriving thread before unparking the pool.
- Thread interruption breaks the barrier, throwing BrokenBarrierException across all parties.

22

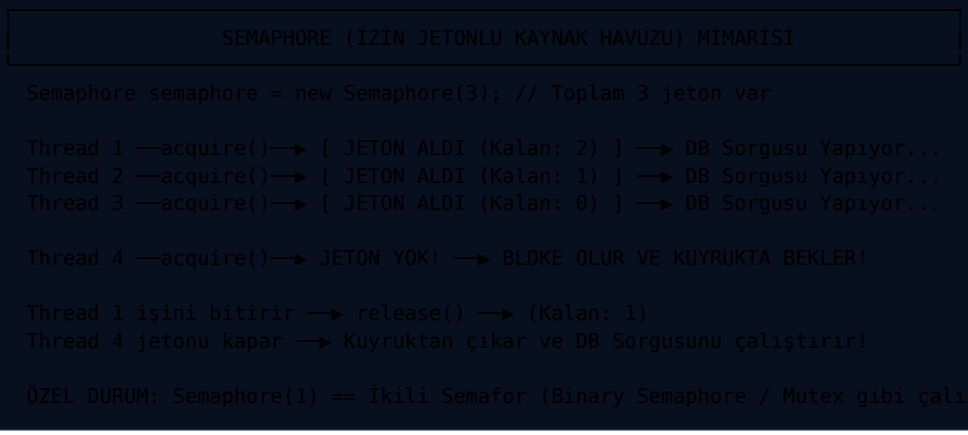
BÖLÜM: İLERI DÜZEY SENKRONIZASYON & KILIT-SIZ ARAÇLAR | MODULE: ADVANCED SYNCHRONIZATION & LOCK-FREE PRIMITIVES

Semaphore: İzin Tabanlı Kaynak Havuzları & Backpressure Sınırlama

Semaphore: Permitted Resource Pooling & Backpressure Throttling

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir gece kulübünün otoparkını düşünün. Otoparkta sadece 3 araçlık yer var (Semaphore permits = 3). İlk gelen 3 araba içeri girer ve güvenlik bariyeri kapanır. 4. araba kapıda bekler. İçerideki arabalardan biri çıkış yaptığı anda (`release()`), kapıdaki arabaya yeşil ışık yanar (`acquire()`) ve içeri girer. Semafor, pahalı bir kaynağı (veritabanı bağlantısı, harici API kotası) aynı anda aşırı yüklenmeden koruyan jetonlu güvenlik turnikesidir!

🇬🇧 ENGLISH (ELI5)

Think of a nightclub coat check with exactly 3 plastic hangers (`Semaphore(3)`). The first 3 patrons receive hangers and enter. The fourth patron waits in line at the counter. The moment an existing patron checks out and returns their hanger (`release()`), the clerk hands it to the waiting patron (`acquire()`). A Semaphore acts as a token-based rate-limiter, throttling concurrent access to bounded resources (database pools, third-party REST quotas) to prevent system brownouts.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Semafor Mimarisi ve Mutex Farkı: 1) AQS State = İzin Sayısı: Semaphore, AQS'in `state` alanını kalan izin (permit) sayısı olarak kullanır. 2) Semaphore vs Mutex (Önemli Mülakat Farkı!): Bir Mutex kilitte kilidi KİM ALDIYSA YALNIZCA O BIRAKABİLİR (Ownership prensibi). Ancak Semaphore'da SAHİPLİK YOKTUR! Thread A `acquire()` çağırıp izni alabilir; Thread B ise `release()` çağırarak o izni havuza geri verebilir! Bu özellik thread'ler arası sinyalleşmede kullanılır. 3) Backpressure ve Rate Limiting: Veritabanına aynı anda en fazla 20 sorgu gitmesini istiyorsanız, arkaya 1000 thread gelse bile `semaphore.tryAcquire(timeout)` ile veritabanı OOM ve çökmeden korunur.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Semaphore Internals & Mutex Distinction: 1) AQS Permit Representation: AQS `state` holds the available permit count. `acquire()` executes a CAS loop decrementing state; if state <= 0, the thread parks on the AQS shared sync queue. 2) Crucial Ownership Difference vs Mutex: A Mutex or `ReentrantLock` has strict ownership: only the thread that acquired the lock may release it. A Semaphore HAS NO CONCEPT OF OWNERSHIP. Thread 1 can invoke `acquire()`, and Thread 2 can invoke `release()`! 3) Dynamic Permit Inflation: Calling `release()` without preceding acquires actually INCREASES permits beyond initial capacity, a feature used in reactive backpressure signaling but a hazardous bug if done accidentally.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Semaphore ile Veritabanı Bağlantı Havuzu Throttling
import java.util.concurrent.Semaphore;
import java.util.concurrent.TimeUnit;

public class DatabaseConnectionThrottler {
    // Veritabanı maksimum 3 eşzamanlı bağlantıya izin veriyor
    private final Semaphore semaphore = new Semaphore(3, true); // Adil (Fair)

    public void executeQuery(String queryName) {
        try {
            // 1 saniye içinde izin alamazsa vazgeç (Fail-Fast)
            if (semaphore.tryAcquire(1, TimeUnit.SECONDS)) {
                try {
                    IO.println("Bağlantı alındı: " + queryName + " yürütülüyor...");
                    Thread.sleep(300); // DB işlemi simülasyonu
                } finally {
                    semaphore.release(); // İzni havuza geri ver
                    IO.println("Bağlantı bırakıldı: " + queryName);
                }
            } else {
                IO.println("⚠ BAĞLANTI HAVUZU DOLU! " + queryName + " reddedildi (Backpressure).");
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'Bir semafora fazladan release() çağrılırsa ne olur?' diye sorar. Yanıt: 'Hata fırlatmaz! İzin sayısı başlangıçtaki kapasitenin üzerine çıkar (Permit Inflation)! Örneğin 3 izinli semafora 5 kere release() derseniz artık 5 thread aynı anda içeri girebilir. Bu durum beklenmedik kaynak tükenmelerine yol açabilir.'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Subtle Interview Trap: 'What happens if you invoke `semaphore.release()` more times than `acquire()`?' Answer: 'No exception is thrown! The permit count increases monotonically past the initial constructor capacity (Permit Inflation). Calling release() 5 times on a Semaphore(3) raises available permits to 5, potentially overwhelming downstream resources.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Semaphore, eşzamanlı erişilebilecek kaynak sayısını (permits) sınırlar.
- Semaforlarda sahiplik (ownership) yoktur; bir thread alıp başka thread bırakabilir.
- İkili Semafor (Binary Semaphore / permits=1), mutex alternatifi olarak kullanılabilir.

🎯 Key Takeaways for the Interview

- Semaphore limits the maximum number of concurrent threads accessing a resource.
- Semaphores have no thread ownership; Thread A can acquire and Thread B can release.
- A binary semaphore (permits=1) functions as a non-reentrant mutual exclusion lock.

23

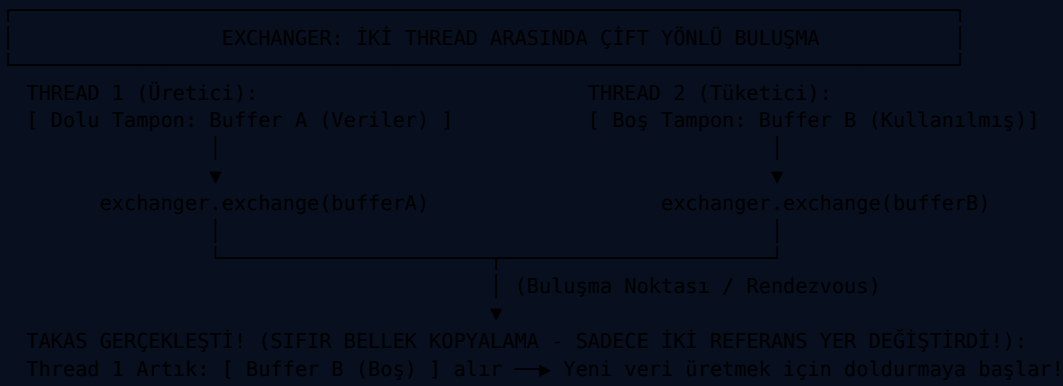
BÖLÜM: İLERI DÜZEY SENKRONIZASYON & KILIT-SIZ ARAÇLAR | MODULE: ADVANCED SYNCHRONIZATION & LOCK-FREE PRIMITIVES

Exchanger: İki Thread Arasında Çift Yönlü Sıfır Kopyalama Veri Takası

Exchanger: Pairwise Zero-Copy Buffer Swapping Between Producer and Consumer

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Casus filmlerindeki köprü sahnesini düşünün. Bir taraftan A ajanı elinde dolu para çantasıyla gelir, diğer taraftan B ajanı elinde boş çantayla gelir. Köprünün ortasında buluşurlar (`exchange()`). Biri gelmeden öbürü çantasını bırakmaz. Tam karşılaştıkları anda çantaları tokuştururlar! A ajanı boş çantayı alıp eve döner, B ajanı para dolu çantayı alıp gider. Bellekte tek bir bayt bile kopyalanmaz; sadece çantalar el değiştirir!

🇬🇧 ENGLISH (ELI5)

Picture a spy bridge exchange. Agent A arrives from the West holding a briefcase packed with cash. Agent B arrives from the East holding an empty briefcase. Neither departs until both arrive at the center span (`exchange()`). At that exact millisecond, they swap cases! Agent A walks away with the empty case to collect more funds; Agent B walks away with the cash. Zero data copying occurs; only two object references instantly swap ownership.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Exchanger Mekanizması ve Ring Buffer Optimizasyonu: 1) Rendezvous (Buluşma) Mantığı: `Exchanger\<V>` iki thread'in çift yönlü senkronize olmasını sağlar. İlk gelen thread, ikinci thread gelene kadar `LockSupport.park()` ile bekletilir. İkincisi geldiğinde referanslar atomik olarak takas edilir ve her iki thread uyandırılır. 2) Sıfır Çöp Üretimi (Zero-Allocation GC Friendly): Yüksek frekanslı finansal sistemlerde (HFT) sürekli `new byte[]` tahsis etmek GC pause yaratır. Exchanger ile önceden ayrılmış iki bellek tamponu (Double Buffering) sürekli el değiştirir; GC baskısı sıfıra iner! 3) Çoklu Çiftler (Arena Desteği): Birden fazla üretici ve tüketici varsa Exchanger içsel olarak bir Arena yapısı kullanarak thread çiftlerini farklı slotlarda eşleştirir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Exchanger Architecture & Cache Locality: 1) Symmetric Rendezvous: Exchanger provides a bidirectional synchronization point. When Thread 1 calls `exchange(V)`, it registers its slot and parks. When Thread 2 arrives, it executes a CAS swapping references, and unparks Thread 1. 2) Eliminating Garbage Collection Churn: High-Frequency Trading (HFT) and telemetry engines avoid constant buffer re-allocations via Double-Buffering. Producer fills Buffer A; Consumer drains Buffer B. At the boundary, they swap references in sub-microsecond time with ZERO heap allocations. 3) Multi-Thread Arena Scaling: Modern JDK Exchangers implement an Arena of slots to eliminate contention when multiple thread pairs exchange concurrently.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Exchanger ile Çift Tamponlu (Double-Buffering) Log Yazıcı
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Exchanger;

public class DoubleBufferingLogger {
    void main() {
        var exchanger = new Exchanger<List<String>>();

        // Üretici Thread (Log toplayan)
        Thread.ofPlatform().name("LogProducer").start(() -> {
            List<String> currentBuffer = new ArrayList<>();
            try {
                for (int i = 1; i <= 5; i++) {
                    currentBuffer.add("Log Mesajı #" + i);
                    Thread.sleep(50);
                }
                IO.println("Üretici: Tampon doldu, takas bekleniyor...");
                currentBuffer = exchanger.exchange(currentBuffer); // Dolu ver, boş al!
                IO.println("Üretici: Boş tampon alındı, boyut: " + currentBuffer.size());
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        // Tüketici Thread (Diske yazan)
        Thread.ofPlatform().name("LogConsumer").start(() -> {
            List<String> emptyBuffer = new ArrayList<>();
            try {
                IO.println("Tüketici: Veri gelene kadar bekleniyor...");
                List<String> fullBuffer = exchanger.exchange(emptyBuffer); // Boş ver, dolu al!
                IO.println("Tüketici: Dolu tampon alındı! Diske yazılıyor: " + fullBuffer);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Kullanım Kuralı: Exchanger tam olarak İKİ thread arasındaki takaslar için tasarlanmıştır. Eğer 3 thread aynı Exchanger'ı kullanmaya kalkarsa, hangi iki thread'in eşleşeceği öngörülemez ve veri akış sırası bozulabilir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Design Limitation: Exchanger is explicitly optimized for pairwise (2-thread) cooperation. Utilizing an Exchanger across odd numbers of threads or uncoordinated pools yields non-deterministic pairing, corrupting ordering protocols.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Exchanger, iki thread arasında çift yönlü ve sıfır kopyalama ile veri takas eder.
- Double-Buffering deseninde GC baskısını sıfırlamak için idealdir.
- İlk gelen thread, ikinci thread gelene kadar park edilerek bekletilir.

🎯 Key Takeaways for the Interview

- Exchanger facilitates bidirectional, zero-copy object swapping between two threads.
- Ideal for Double-Buffering architectures to eliminate GC allocation churn.
- The first arriving thread parks until the second arrives, ensuring atomic handoff.

24

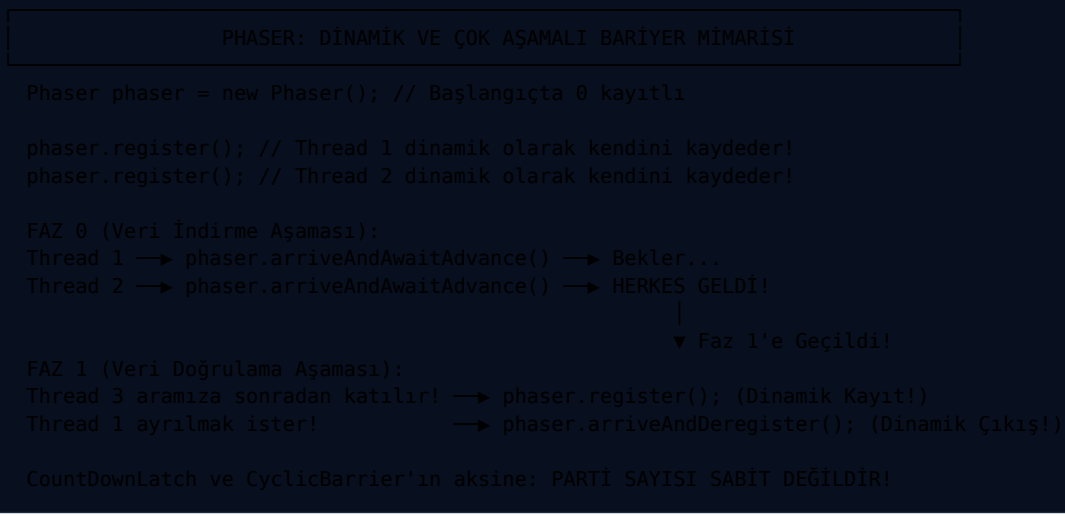
BÖLÜM: İLERI DÜZEY SENKRONIZASYON & KILIT-SIZ ARAÇLAR | MODULE: ADVANCED SYNCHRONIZATION & LOCK-FREE PRIMITIVES

Phaser: Dinamik Thread Kaydı & Çok Fazlı İleri Düzey Bariyer

Phaser: Dynamic Thread Registration & Multi-Phase Barrier Architecture

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir maraton düşünün. Ama bu maratonda yarışmacı sayısı baştan belli değildir! Koşu sırasında yeni koşucular yarışa dahil olabilir (`register()`), yorulan koşucular yarıştan çekilebilir (`arriveAndDeregister()`). Maraton 3 etaptan oluşur: 1. Etap (Faz 0), 2. Etap (Faz 1), 3. Etap (Faz 2). Her etabın kontrol noktasına gelen koşucu diğerlerini bekler; herkes geldiğinde hakem düdüğü çalar ve bir sonraki etap başlar! Phaser, katılımcı sayısı dinamik değişen çok etaplı bir yarış hakemidir!

🇬🇧 ENGLISH (ELI5)

Imagine a multi-stage endurance race where the participant roster is fluid. Mid-race, new runners can dynamically register (`register()`), and injured runners can withdraw (`arriveAndDeregister()`). The race spans discrete stages: Stage 0 (Mud Run), Stage 1 (Swim), Stage 2 (Sprint). At each checkpoint, runners wait until all registered participants arrive; the referee then advances the phase counter. Phaser is a dynamic, multi-phase CyclicBarrier supporting variable party counts.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Phaser vs CyclicBarrier / CountDownLatch: 1) Dinamik Katılımcı Sayısı (Dynamic Parties): CyclicBarrier'da katılımcı sayısı `new CyclicBarrier(N)` ile sabittir ve değiştirilemez. Phaser'da ise çalışma anında thread'ler `register()`, `bulkRegister(n)` veya `arriveAndDeregister()` ile serbestçe katılıp ayrılabilir. 2) Çok Fazlı Yapı (Phases): Phaser içsel olarak `getPhase()` ile artan bir faz sayacı tutar (0, 1, 2...). 3) Ağaç Yapılı Ölçekleme (Tiered Phasers): Yüzlerce thread'in tek bir senkronizöre saldırmasını önlemek için Phaser'lar hiyerarşik (Parent-Child) ağaçlar halinde kurulabilir; bu da devasa çok çekirdekli sunucularda çekişmeyi azaltır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Phaser Architecture & Scalability: 1) Dynamic Party Mutation: Unlike CyclicBarrier, party counts are mutable. Threads register on the fly (`register()`) and deregister upon early task completion (`arriveAndDeregister()`), preventing abandoned barriers. 2) Phase Counter Progression: Phaser tracks phase numbers monotonically via a 64-bit state field. `onAdvance(int phase, int registeredParties)` can be overridden to execute barrier logic or terminate the phaser. 3) Hierarchical Tree Phasers: To overcome memory bus contention across hundreds of cores, Phasers support hierarchical parent-child configurations, reducing synchronization contention via subtrees.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Phaser ile Dinamik Katılımlı Çok Fazlı İş Akışı
import java.util.concurrent.Phaser;

public class MultiPhaseTaskRunner {
    void main() {
        var phaser = new Phaser(1); // Main thread kendini kaydeder (Party: 1)

        for (int i = 1; i <= 2; i++) {
            phaser.register(); // Dinamik olarak yeni worker kaydet
            final int workerId = i;
            Thread.ofPlatform().name("Worker-" + workerId).start(() -> {
                IO.println("Worker " + workerId + " | Faz 0 (Veri İndirme) başladı.");
                phaser.arriveAndAwaitAdvance(); // Faz 0 tamamlandı, bekle!

                IO.println("Worker " + workerId + " | Faz 1 (Veri İşleme) başladı.");
                phaser.arriveAndAwaitAdvance(); // Faz 1 tamamlandı, bekle!

                phaser.arriveAndDeregister(); // İş bitti, kaydı sil ve ayrıl!
            });
        }

        IO.println("Main: Faz 0 bekleniyor...");
        phaser.arriveAndAwaitAdvance(); // Main thread Faz 0'ı onaylar

        IO.println("Main: Faz 1 bekleniyor...");
        phaser.arriveAndAwaitAdvance(); // Main thread Faz 1'i onaylar

        phaser.arriveAndDeregister(); // Main ayrılır
        IO.println("Tüm fazlar başarıyla tamamlandı!");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Sorusu: 'CyclicBarrier varken neden Phaser kullanmalıyız?' Yanıt: '1) Thread sayısı dinamik değişiyorsa, 2) Görevini erken bitiren thread'lerin havuzdan düşmesi gerekiyorsa, 3) Hiyerarşik ağaç yapısıyla binlerce thread'e ölçeklenmek gerekiyorsa Phaser tercih edilir.'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Interview Comparative Analysis: 'Why select Phaser over CyclicBarrier?' Senior Answer: '1) Dynamic thread registration/deregistration at runtime. 2) Programmable termination conditions via `onAdvance()`. 3) Tiered hierarchical tree structures to mitigate cache coherency contention under extreme thread counts.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Phaser, çalışma anında dinamik thread kaydına (register/deregister) izin verir.
- getPhase() ile aşama aşama ilerleyen çok adımlı iş akışlarını koordine eder.
- Hiyerarşik (Parent-Child) Phaser yapıları binlerce çekirdeğe ölçeklenebilir.

🎯 Key Takeaways for the Interview

- Phaser supports dynamic registration and deregistration of parties at runtime.
- Tracks sequential workflow phases via an incrementing phase counter.
- Hierarchical phasers eliminate bus contention across massive core counts.